

League Objective Setup Analytics

What separates good and bad objective setups in League of Legends, and how that varies by rank, objective type, and game state.

3,839

MATCHES ANALYZED

34,167

OBJECTIVE INSTANCES

Core research question

What conditions make contesting, starting, trading, or giving an objective the right call? The project focuses on Dragon and Baron first.

What this project does

- Builds a clean objective-window table, one row per team per objective
- Labels outcomes beyond just secured / not secured
- Separates the game state at T-60 from what teams actually chose to do by T-30
- Compares teams in similar situations to see how much setup choices matter

Methods

Unit of analysis

Each row is one team's perspective on one objective instance.

```
match_id + objective_instance + team_id → one row
```

For each objective two rows are created, one per team side. Features describe only that team's state relative to the opponent.

Objective time windows

Features are computed within windows relative to T=0 (objective take time).

T-120 → T-90

Early setup

T-90 → T-60

State (pre-commitment)

T-60 → T-30

Setup actions

T-30 → T+30

Contest / take

T+30 → T+120

Aftermath

State-first analytical framework

Separate what was already true at T-60 from what teams chose to do between T-60 and T-30.

01

State

Information at T-60 or earlier. Gold difference, alive counts, recent deaths.

02

Action

Observed behaviour between T-60 and T-30. Who arrived first, how many champions appeared near the objective, whether setup was contested or conceded.

03

Outcome

Objective secured, outcome label, net objective value. Never used as a model feature.

Comparing within similar T-60 states is cleaner than looking across all games at once because the starting situation is roughly controlled for. Still observational, not causal.

State bucket definitions

The T-60 state is broken into three dimensions that get combined into one bucket label.

Gold state (T-60)

Gold lead as % of estimated team gold at game time, split into quintiles from the data.

— **big_behind**: bottom 20%

— **behind**: 20th-40th percentile

— **even**: middle 20% (40th-60th)

— **ahead**: 60th-80th percentile

— **big_ahead**: top 20%

Normalizing by estimated team gold keeps thresholds comparable across Dragon 1 and Dragon 2.

Alive state (T-60)

— **numbers_down**: fewer alive than enemy

— **even_alive**: equal alive counts

— **numbers_up**: more alive than enemy

Death state

(standalone feature)

Kept as a standalone feature but not part of the state bucket. It overlaps heavily with alive_state, and adding it would create 60 bucket combinations instead of 15.

— **no_recent_deaths**: neither side lost a champion in the last 90s

— **enemy_pick**: enemy had deaths, team did not

— **team_pick**: team had deaths, enemy did not

— **trade_deaths**: both sides had deaths

The combined bucket reads as gold_state | alive_state, e.g. 'even | numbers_up'. That gives 15 combinations. Adding death state would push it to 60, which leaves too few rows per bucket to compare meaningfully.

Setup profile definitions

Profiles classify each team's T-30 posture relative to the opponent.

free setup

Team present, enemy absent, no recent allied deaths.

Full positional control.
Secure rate is highest here.

free setup deaths

Team present, enemy absent, but allied deaths in the prior 60s.

Uncontested positionally but down a player or two.

clean contest

Both teams near objective, team not short-handed or behind on health.

Even or favorable fight with both sides present.

disadvantaged

Both teams present, but team had recent deaths or fewer

gave away

Enemy present at objective, team absent.

both absent

Neither team near objective at T-30.

alive champions.

Contesting from behind.

Team did not show up.

Neither side committed.

One team eventually took it.

Outcome label definitions

Ten outcome categories describe what happened at and after the objective.

clean take

Team secured. Won the fight or there was no real fight.

clean give

Enemy secured. Team did not contest, which may have been intentional.

good trade

Team gave the objective but picked up meaningful value elsewhere.

coinflip

Both teams showed up with no clear advantage. Outcome was roughly 50/50.

bad contest

Team contested from a weak spot (short-handed, recent deaths) and lost.

won fight lost objective

Team won the fight but still lost the objective.

lost fight got objective

Team lost the fight but secured the objective anyway.

throw setup

Team was in good shape but lost a player in the last 30s before the fight.

objective steal

Team secured from a clearly outnumbered position, usually via smite.

no meaningful contest

Neither team committed. Objective taken without a real fight.

Modeling approach

Models are used as analytical tools, not prediction engines.

- 1 Descriptive statistics and setup profiles
- 2 State-only logistic regression (T-60 features only)
- 3 Setup-only logistic regression (T-30 action features only)
- 4 Combined model (state + setup): measures how much setup adds on top of state
- 5 State-conditioned within-bucket comparisons for each action

Train/test splits are grouped by match_id. All rows from one match stay together in train or test, which prevents leakage from objectives within the same game.

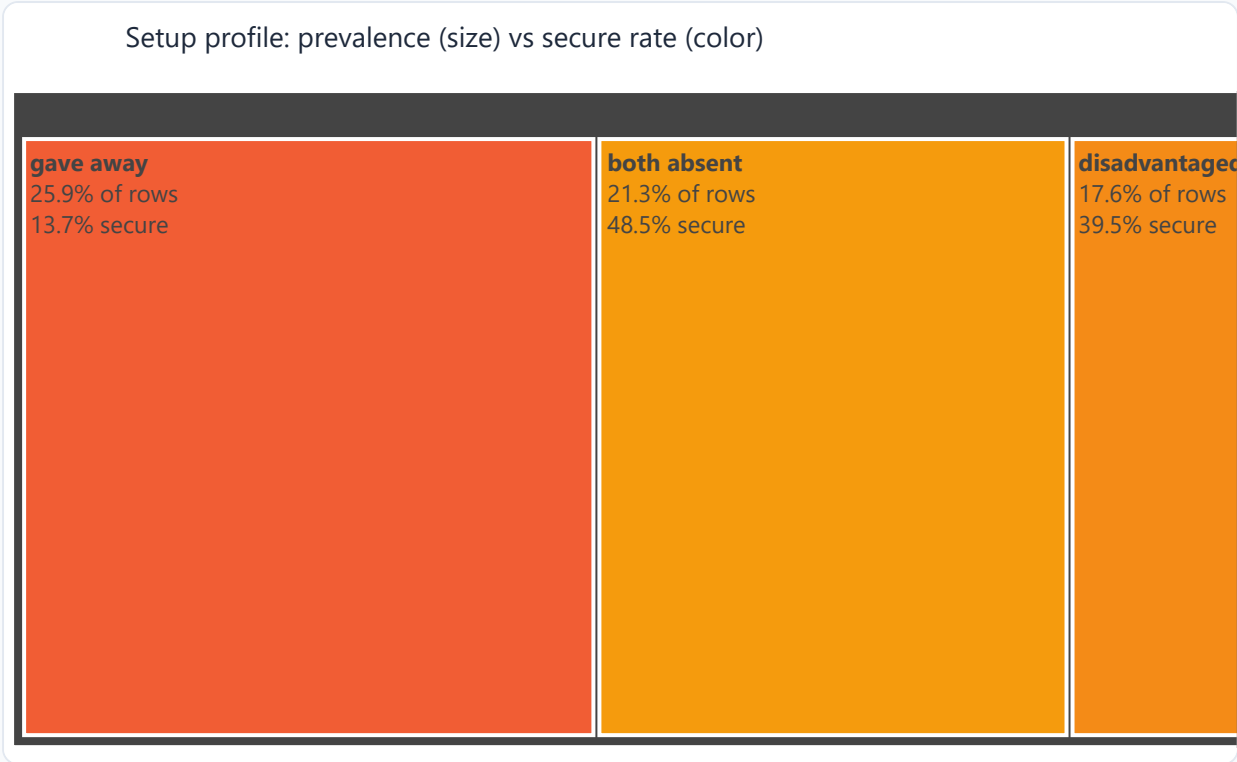
Data and limitations

- Data source: Riot Games Match-V5 API (match summaries and timeline events)
- Exact player pathing, wave state, and item timing are not available, so proxies are used
- Role assignment is inferred from position data and may have errors for off-meta compositions
- Findings are observational. Teams that arrive first may do so because of champion advantages the model cannot see
- Language throughout: 'associated with', 'predictive of', 'proxy for'. Not 'proves' or 'causes'

Analysis & Results

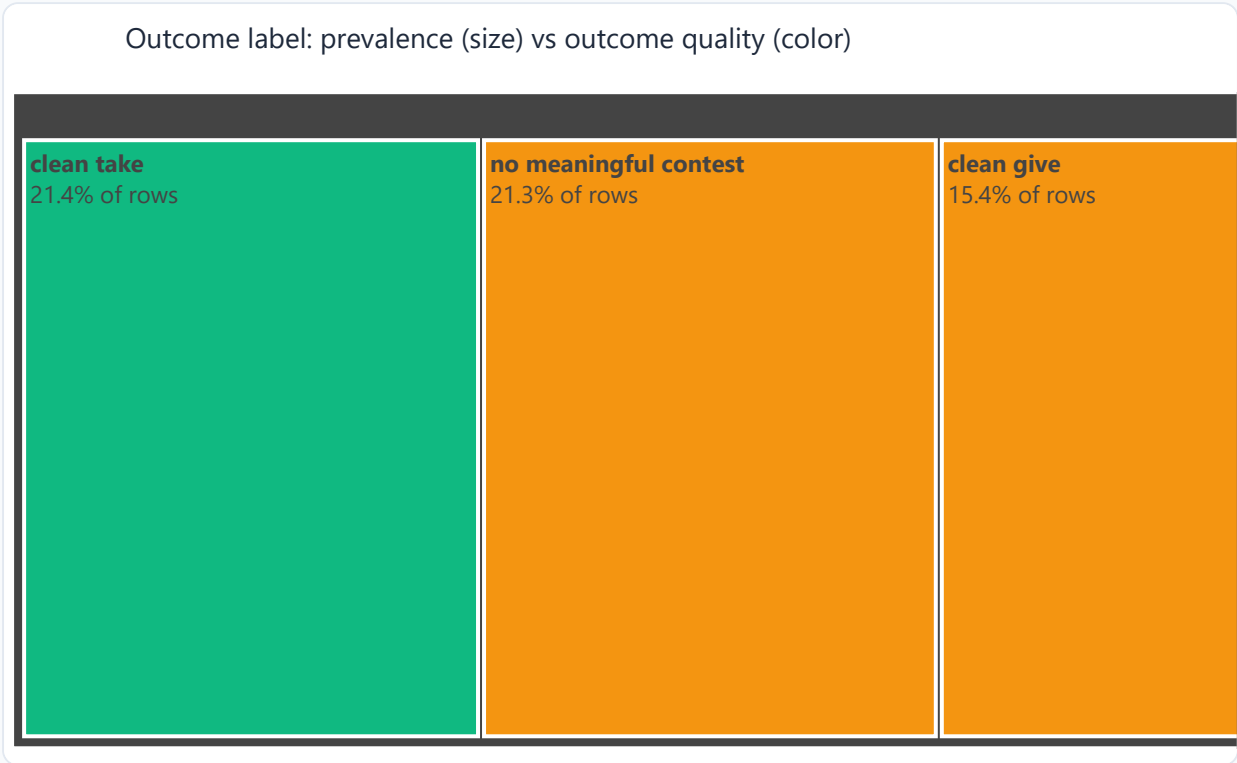
Setup profiles: frequency and secure rate

Tile size = prevalence. Color = secure rate (red to green).



Outcome labels: prevalence and strategic quality

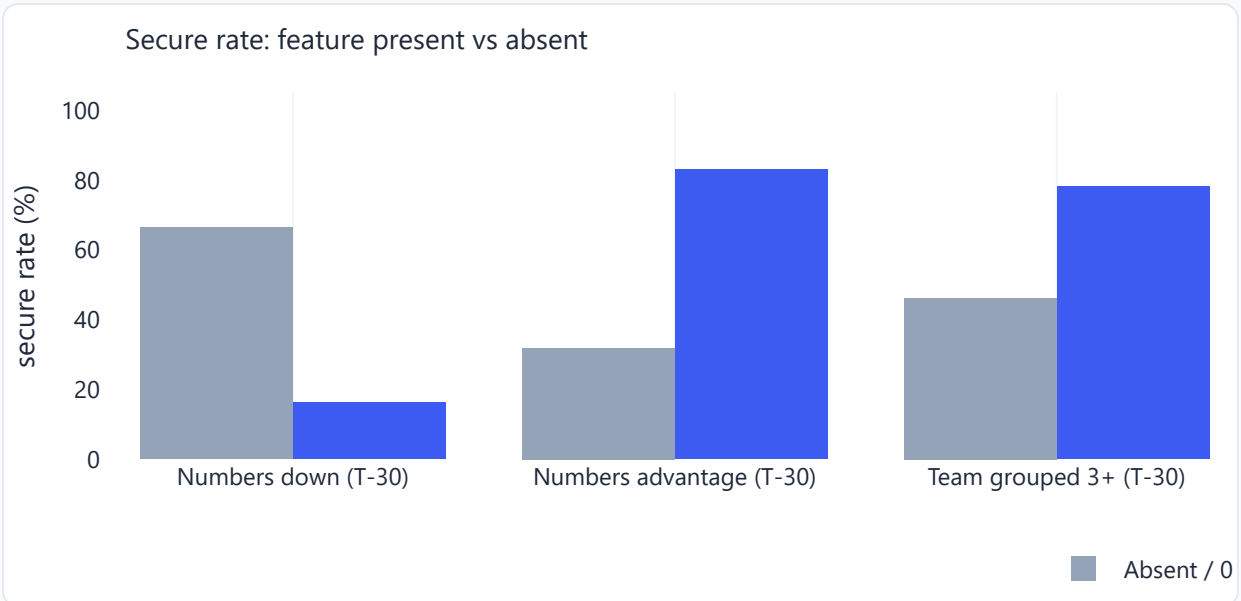
Tile size = frequency. Color = strategic quality.



lost_fight_got_objective is more common than won_fight_lost_objective because if you win the fight you almost always have time to take the objective too.

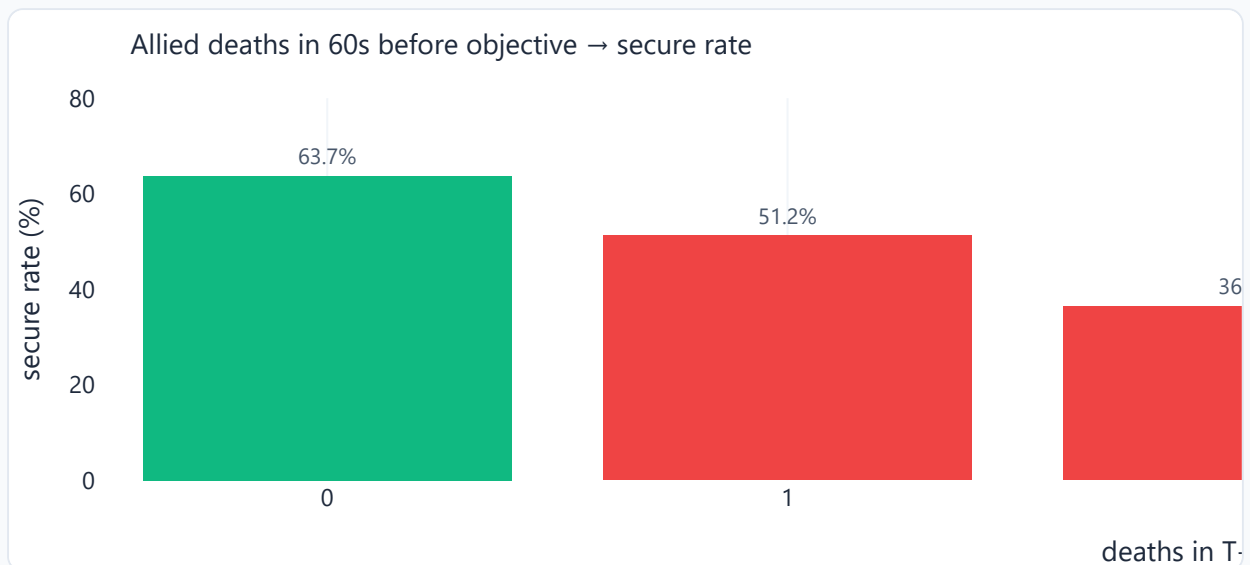
Impact of key binary features

Secure rate with and without each binary feature.



Deaths before objective

Each allied death in the 60s before the objective substantially reduces the secure rate.

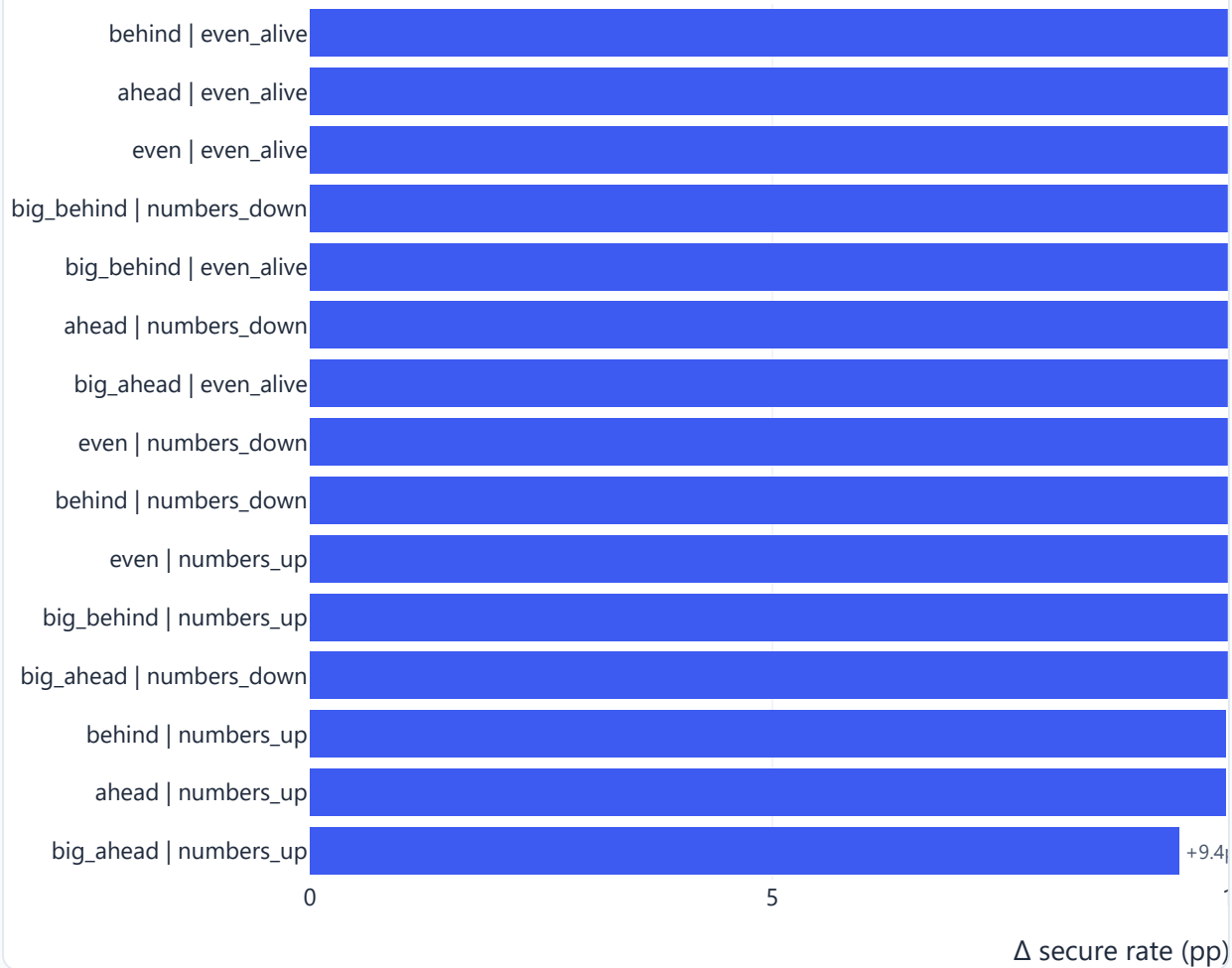


Even one death drops the secure rate by roughly 20 percentage points.

State-first: value of arriving first

Within each T-60 state bucket, teams that arrived first vs those that did not.

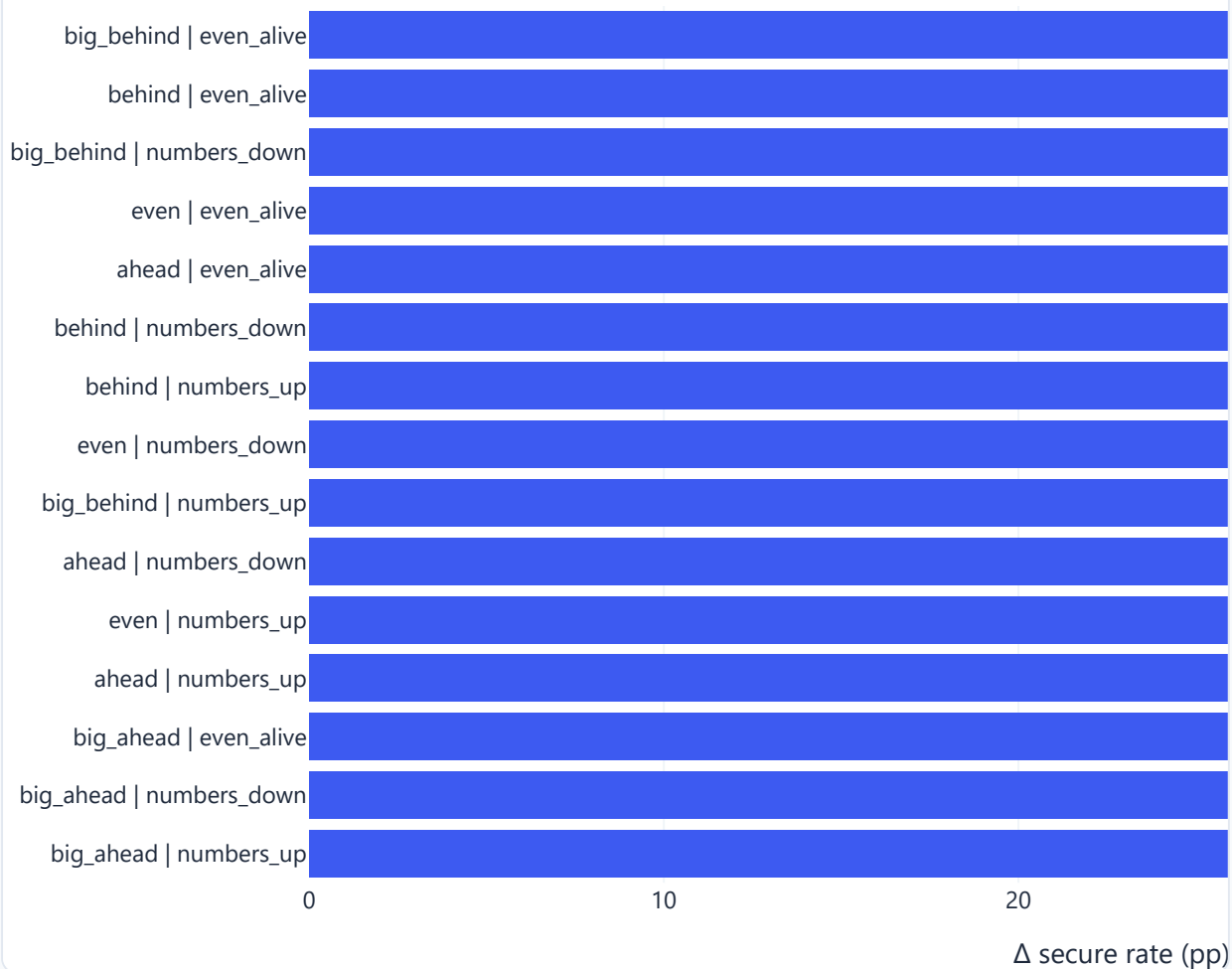
State-conditioned effect: arrived first



State-first: value of numbers advantage at T-30

Same analysis for having more allies than enemies near the objective at T-30.

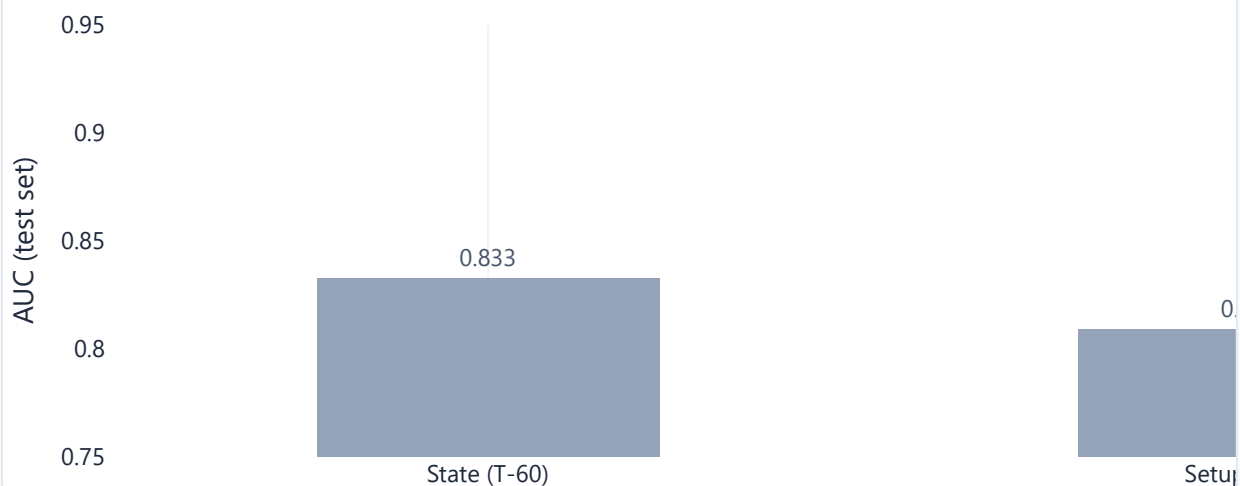
State-conditioned effect: numbers advantage at T-30



Model comparison

State + Setup improves AUC by 5.9 pp over State alone.

Model AUC: state vs setup vs combined

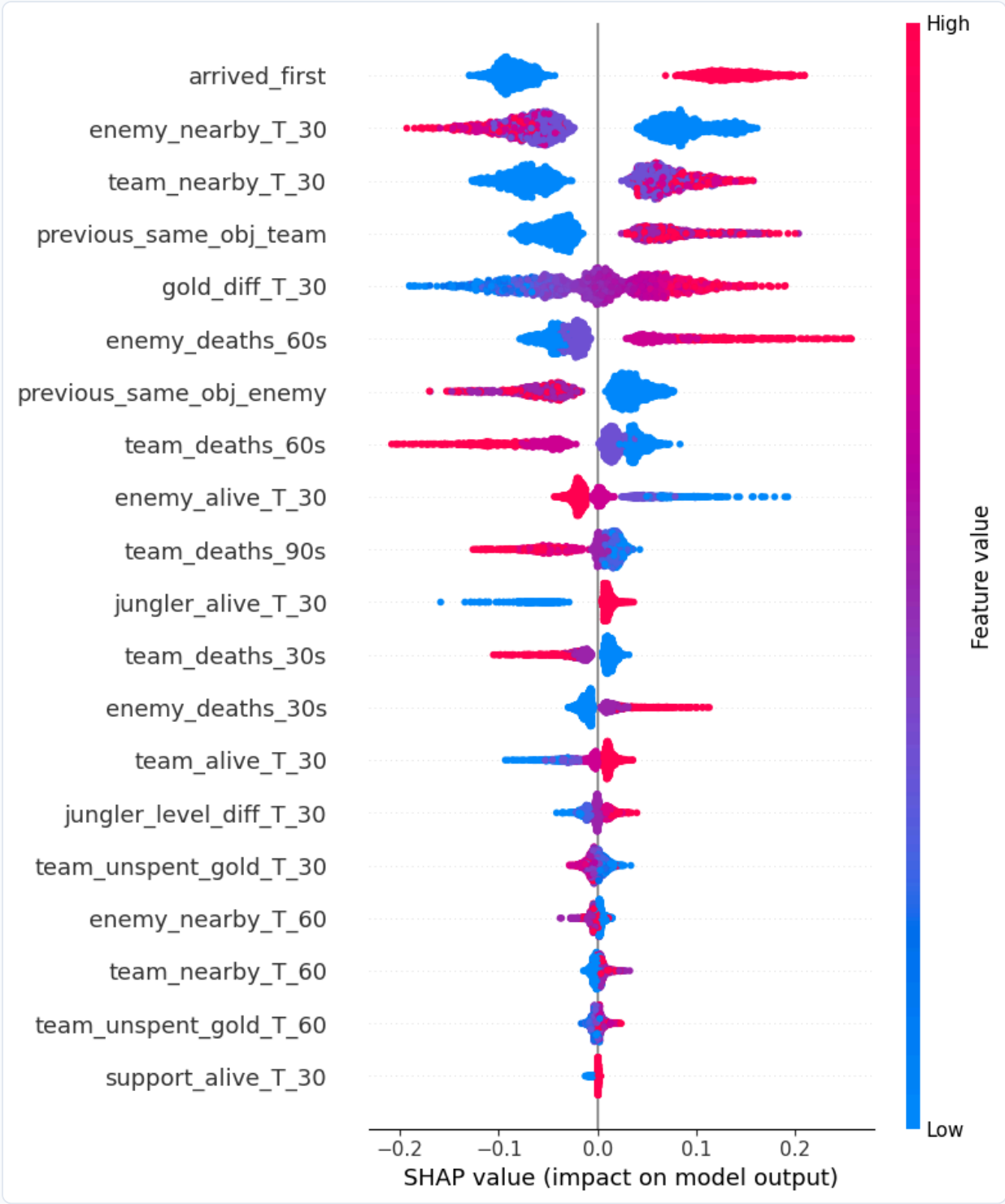


MODEL	AUC	LOG-LOSS	BRIER	FEATURES
State (T-60)	0.8327	0.5049	0.1669	29

MODEL	AUC	LOG-LOSS	BRIER	FEATURES
Setup (T-30)	0.8091	0.5234	0.1737	7
State + Setup	0.8916	0.4177	0.1320	36

Feature importance (SHAP)

Random Forest model. Each dot is one objective row.



Outcome distribution by rank

How outcomes differ across skill tiers.

